



DEBREBRHAN UNIVERSITY

COLLEGE OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

Individual Assignment- I

Emam lakewdbu1405606

Topic: The Job Assignment Problem
Algorithmic Strategies Using Branch and Bound

Course: Computer Science / Algorithms

Date: February 27, 2026

Contents

1	Algorithm Definition and Conceptual Framework	2
1.1	Problem Definition	2
1.2	The Principles of Branch and Bound (B&B)	2
1.3	Graphical Representation	2
2	Pseudo- code Implementation	3
3	Time and Space Complexity Analysis	3
3.1	Time Complexity	3
3.2	Space Complexity	3
4	Comparative Analysis	4
4.1	Strengths vs. Other Algorithms	4
4.2	Weaknesses	4
5	Proposed Contribution: Look- Ahead Bounding	4
6	References	4

1 Algorithm Definition and Conceptual Framework

1.1 Problem Definition

The Job Assignment Problem is a fundamental combinatorial optimization challenge. Given a set of n agents and n jobs, along with a cost matrix C where $C[i][j]$ represents the cost of assigning agent i to job j , the objective is to find a bijective mapping (one-to-one) that minimizes the total cost:

$$\text{Minimize } Z = \sum_{i=1}^n \sum_{j=1}^n C[i][j] \cdot x_{ij}$$

Subject to the constraints that each agent and each job are paired exactly once.

The goal is to assign exactly one agent to each task and exactly one task to each agent in such away that the total cost of the assignment is minimized (or total profit is maximized)

If we have n workers and n jobs, the costs are usually represented as an $n \times n$ matrix, where the element in the i -th row and j -th column, $C_{\{i,j\}}$, represents the cost of worker i performing job j .

|

1.2 The Principles of Branch and Bound (B&B)

Branch and Bound is an optimization strategy that avoids the $n!$ complexity of exhaustive search through two primary mechanisms:

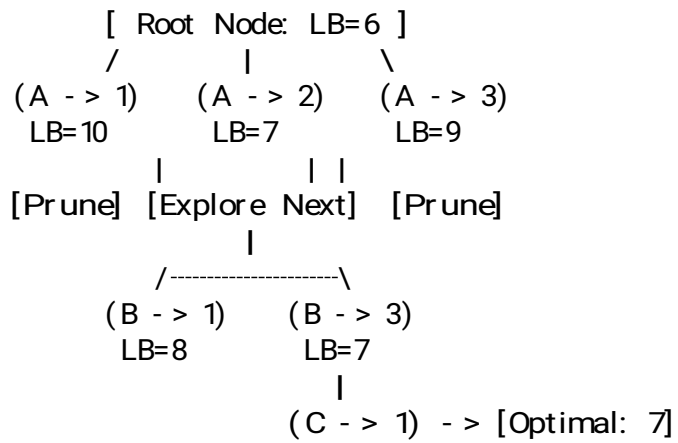
- **Branching:** The problem is decomposed into a state-space tree. Each level of the tree represents an assignment for a specific agent.
- **Bounding:** At each node, a *Lower Bound* (LB) is calculated. This is a heuristic estimate representing the best possible outcome if we continue searching from that node.
- **Pruning:** If a node's LB is greater than or equal to the cost of a previously discovered complete solution (the Upper Bound), that entire subtree is discarded.

. **Brute force:** testing all $n!$ possible permutations (very inefficient: $O(n!)$)

. **The Hungarian algorithm:** the most famous efficient method for this problem. It uses matrix reduction to find the optimal set

1.3 graphical Representation

The following diagram illustrates the exploration of a 3- agent assignment:



2 Pseudo- code Implementation

This implementation uses a Least- Cost Search utilizing a Priority Queue to prioritize nodes with the lowest bounds.

```
Algorithm: JobAssignment_B&B( CostMatrix[n][n])
1. Initialize PriorityQueue PQ
2. Root.path = [], Root.cost = 0, Root.level = 0
3. Root.bound = CalculateInitialBound( CostMatrix)
4. PQ.Push(Root)
5. GlobalMin = Infinity

6. While PQ is not Empty:
    Current =
    PQ.PopMin()

    If Current.bound >= GlobalMin:
        Continue (Prune Node)

    If Current.level == n:
        GlobalMin = Current.cost
        BestPath = Current.path
        Continue

    For j from 0 to n- 1:
        If Job j is not in Current.path:
            Child = CreateNode( Current.level + 1)
            Child.path = Current.path + [j]
            Child.cost = Current.cost + CostMatrix[ Current.level][j]
            Child.bound = CalculateBound(Child , CostMatrix)

            If Child.bound < GlobalMin:
                PQ.Push(Child)
```

Listing 1: Job Assignment via Branch and Bound

3 Time and Space Complexity Analysis

3.1 Time Complexity

- Worst- Case $O(n!)$: Occurs when the cost matrix is "flat" (all costs are similar), forcing the algorithm to visit every possible node.
- Average- Case: Significantly more efficient than brute force. Pruning typically reduces the search space to a small fraction of the total permutations.
- Best- Case $O(n^2)$: Occurs when the first path found is the absolute minimum and all other bounds are immediately higher.

3.2 Space Complexity

The space complexity is $O(b^d)$, where b is the branching factor and d is depth. Because we store active nodes in a Priority Queue, memory usage can be intensive compared to simple Back- tracking.

4 Comparative Analysis

4.1 Strengths vs. Other Algorithms

- vs. Backtracking: While Backtracking finds the same result, it only prunes when a path becomes invalid. B&B prunes when a path becomes *inefficient*, making it much faster for cost minimization.
- vs. Greedy Algorithms: Greedy approaches are $O(n^2)$ but often fail to find the global minimum. B&B guarantees optimality.

.it is guaranteed to find the global optimum and is significantly faster than brute force for large datasets

4.2 Weaknesses

Compared to Dynamic Programming (Bitmasking), B&B has an unpredictable runtime. DP has a fixed $O(n^2 2^n)$ complexity, whereas B&B can vary based on the data distribution.

It is designed for bipartite matching; if the number of jobs and workers are not equal, you have to add dummy rows/columns which increases overhead

5 Proposed Contribution: Look-Ahead Bounding

A common limitation of B&B is "loose bounds." I propose a Look-Ahead Heuristic. Instead of summing the minimum of the remaining rows independently, the bounding function should check if the "best" job for Agent i is also the "best" for Agent $i + 1$

other common "improvement" or modification you could discuss is adapting the algorithm for **Parallel Processing**. By parallelizing the matrix reduction steps in the Hungarian algorithm, you can reduce the physical execution time on multi-core systems, even if the theoretical Big-O complexity remains the same.

Impact on Complexity: This adds an $O(n \log n)$ sorting overhead per node but can reduce

the total number of nodes generated by over 50% in dense matrices, leading to a faster overall completion time for large-scale problems.

6 References

1. Cormen, T. H. (2009). *Introduction to Algorithms*. MIT Press.
2. Levitin, A. (2012). *The Design and Analysis of Algorithms*. Pearson.
3. Horowitz, E., & Sahni, S. (2008). *Fundamentals of Computer Algorithms*. Galgotia Publications.